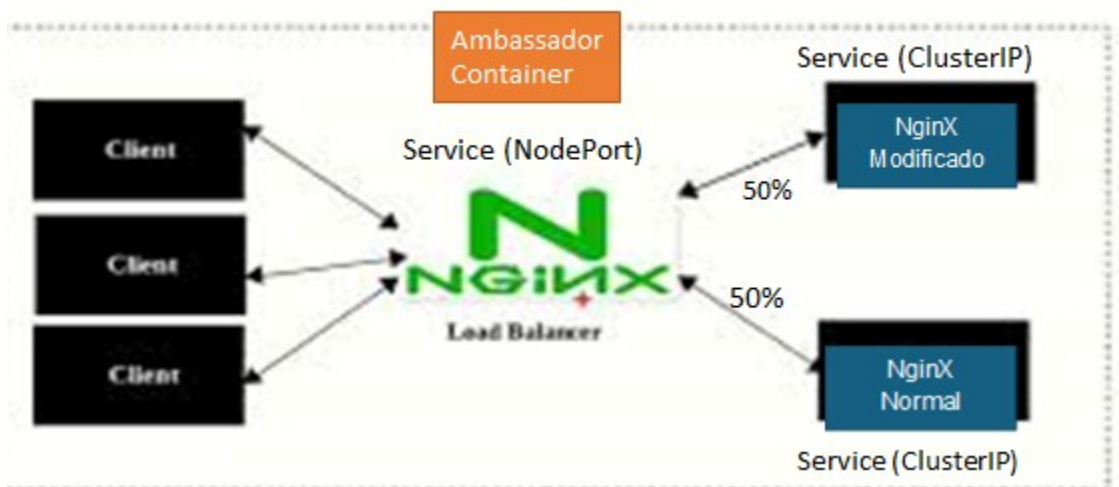
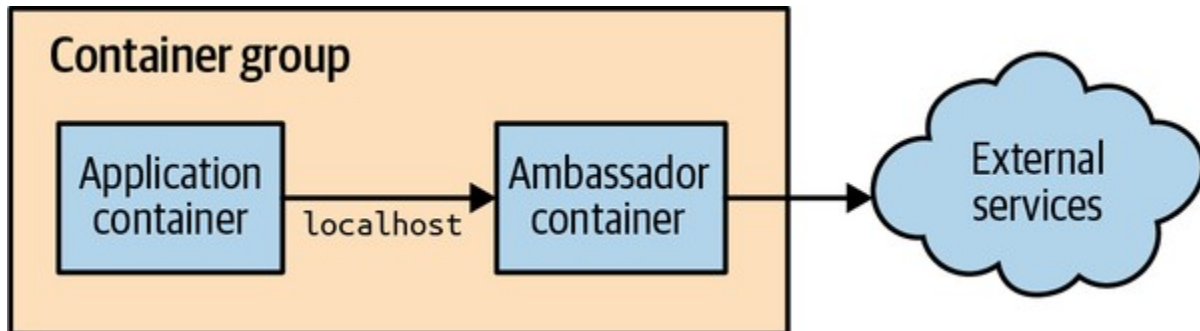


Patrón: Ambassador

En este ejemplo se usa el patrón Ambassador para hacer request splitting.

El ejemplo es **Hands On: Implementing 10% Experiments** (Libro:) el ejercicio tiene algunas modificaciones con respecto a lo sugerido en el libro. La arquitectura se observa a continuación:



Conceptos esenciales para entender el ejercicio:

- ConfigMaps
- Deployments
- Services
- ClusterIP y Node Port

Archivos necesarios:

web.yaml, experiment.yaml, miambassador2.yaml, configmap.yaml, configmap1.yaml, prueba1.py

Notas sobre la ejecución: el ejercicio fue probado en minikube.

Cómo se ejecuta el taller:

1. Iniciar Minikube
2. Crear el primer configmap (el de experiment)

```
kubectl apply -f configmap.yaml
```

Con este comando se crea un configmap llamado nginx-index-conf que se usa para experiment

3. Para observar los configmap creados pueden hacer:

```
kubectl get cm
```

4. Desplegar los dos servicios:

```
kubectl apply -f web.yaml
```

```
Kubectl apply -f experiment.yaml
```

Resultado

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
experiment	ClusterIP	10.103.96.217	<none>	80/TCP	42s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	34d
web	ClusterIP	10.111.91.95	<none>	80/TCP	35s

- Comprobar siempre que los pods se esten ejecutando.
5. Crear el configmap a partir del archivo **configmap1.yaml** **ejecutando el siguiente comando:**

```
kubectl apply -f configmap1.yaml
```

Este comando crea el config map llamado nginx-config, que será utilizado por el último servicio que vamos a levantar.

6. Ejecutar el siguiente comando para levantar el NginX que distribuirá el tráfico entre los dos servicios anteriores:

```
kubectl apply -f miambassador2.yaml
```

Nota: con `kubectl get pods` puede revisar si los pods se están ejecutando correctamente.

Ejecute

`kubectl get svc` // para observar los servicios que se están ejecutando

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
experiment 10m	ClusterIP	10.103.96.217	<none>	80/TCP	
kubernetes 34d	ClusterIP	10.96.0.1	<none>	443/TCP	
my-service 113s	NodePort	10.104.18.186	<none>	8080:32611/TCP	
web 10m	ClusterIP	10.111.91.95	<none>	80/TCP	

Note que el último servicio es del tipo NodePort

Si está dentro de Minikube puede usar el siguiente comando para invocar el servicio y observar cómo se distribuye la carga entre web y experiment.

```
minikube service Nombre-Servicio --url
```

En mi computador me dio este resultado: `http://192.168.49.2:32611`

Esta es la dirección que se va a colocar en el programa **prueba1.py** antes de ejecutarlo. Cuando ejecute el programa podrá observar que las peticiones se distribuyen en forma equitativa entre un servidor nginx y el otro.

Realizado por M. Curiel. Marzo 2026.